

Digital Filters

Script: Chapter 10 (FIR and IIR filter design, filter structures)

with solutions**Learning goals**

- Turn a verbal requirement (“extract the knock resonance”) into a quantitative filter specification and estimate the necessary order.
- Design linear-phase FIR filters with the window method and IIR filters with the bilinear transform (including pre-warping); realise IIR filters as biquad cascades.
- Judge the effects of finite word length: coefficient quantisation, scaling, overflow, saturation; Q15 arithmetic on an 8-bit controller.
- Run the knock band-pass sample by sample in real time on the ESP32 (float) and on the Arduino Uno (fixed point) and compare FIR and IIR in terms of cost and delay.

Background and conventions

The knock sensor signal of the virtual engine contains the knock resonance at $f_1 = 6.5$ kHz (plus a weaker mode at 10.5 kHz), valve impacts at 8 kHz, a low-frequency combustion vibration and broadband noise. The first block of every knock detector is a band-pass filter around f_1 . On this sheet we use

sampling rate	$f_s = 25$ kHz ($T_s = 40$ μ s)
pass band	6.0 kHz to 7.0 kHz, loss at most $A_p = 1$ dB
stop bands	$f \leq 5.0$ kHz and $f \geq 8.0$ kHz, attenuation at least $A_s = 40$ dB

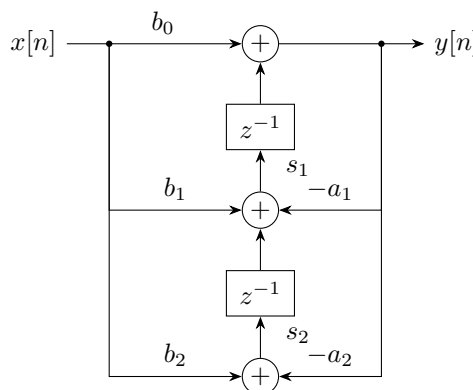
A biquad (second-order section) is written

$$H_k(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}},$$

a cascade is $H(z) = \prod_k H_k(z)$. Two structures are used: *direct form I* (DF I, states $x[n-1], x[n-2], y[n-1], y[n-2]$, one accumulator – preferred in fixed point) and the *transposed direct form II* (TDF II, two states s_1, s_2 – preferred in floating point):

$$y = b_0 x + s_1, \quad s_1 \leftarrow b_1 x - a_1 y + s_2, \quad s_2 \leftarrow b_2 x - a_2 y.$$

$Q15$ denotes a 16-bit two’s complement number $v = q/2^{15} \in [-1, 1)$, $Q2.14$ a 16-bit number $v = q/2^{14} \in [-2, 2)$.



Biquad in transposed direct form II.

Exercise 10.1 Design by hand**PAPER**

- a) *Sampling rate.* The engine-sim chip updates its analog outputs every $20\text{ }\mu\text{s}$ (50 kHz, zero-order hold). Explain why the ESP32 samples the knock signal at $f_s = 25\text{ kHz}$ and not, e.g., at 40 kHz. Estimate the worst-case phase error of the 6.5 kHz component caused by the timing mismatch at 40 kHz. Is 25 kHz sufficient for both knock modes and for the valve disturbance? What happens to the white noise of the chip (generated at 50 kHz) when only every second value is sampled?
- b) *FIR order.* Normalise the specification ($\Delta F = \Delta f/f_s$). Estimate the length N of a window-design FIR filter for a Hamming window ($N \approx 3.3/\Delta F$) and with Kaiser's formula $N - 1 \approx (A_s - 8)/(2.285 \Delta\omega)$, $\beta = 0.5842(A_s - 21)^{0.4} + 0.07886(A_s - 21)$. Where do you put the cut-off frequencies of the ideal band-pass?
- c) Derive the impulse response $h_d[m]$ of the ideal (zero-phase) band-pass with cut-offs $\omega_{c1} < \omega_{c2}$, and state how the causal FIR filter $h[n]$, $n = 0, \dots, N - 1$, is obtained from it. What is its group delay in samples, in ms, and in degrees crank angle at 3000 rpm and 6000 rpm? Why does this matter for a knock measurement window of 10° – 70° after TDC?
- d) *IIR order.* Pre-warp the band edges, $\Omega = 2f_s \tan(\pi f/f_s)$, and compute the stop-band frequency $\lambda_s = \min |\Omega_s^2 - \Omega_0^2|/(B \Omega_s)$ of the normalised low-pass prototype ($\Omega_0^2 = \Omega_{p1}\Omega_{p2}$, $B = \Omega_{p2} - \Omega_{p1}$). Which Butterworth prototype order n is needed, how many biquads does the band-pass have, and how many multiplications per sample does it cost compared with the FIR filter?
- e) *Second-order band-pass.* The knock burst decays with $\tau = 0.8\text{ ms}$. Show that its spectrum has a -3 dB bandwidth of $B_k = 1/(\pi\tau)$ and argue why a band-pass with about this bandwidth is a good compromise (matched filter). Design $H(s) = Bs/(s^2 + Bs + \Omega_0^2)$ with centre $f_0 = 6.5\text{ kHz}$ and digital bandwidth 400 Hz via the bilinear transform $s = 2f_s(1 - z^{-1})/(1 + z^{-1})$ with pre-warping (use $B \approx 2\pi \cdot 400\text{ Hz}/\cos^2(\pi f_0/f_s)$). Compute b_0, b_1, b_2, a_1, a_2 , the pole radius r and the decay time of the filter's impulse response.

Solution

a) The chip output is a staircase on a $20\text{ }\mu\text{s}$ grid. With $f_s = 25\text{ kHz} = 50\text{ kHz}/2$ every ADC sample sees a fresh chip value, always at the same position within the $20\text{ }\mu\text{s}$ step; the sequence is exactly the model signal sampled uniformly at 25 kHz. At 40 kHz the sampling instants slide over the grid: some chip values are read twice, others never, and the value read is up to $20\text{ }\mu\text{s}$ old. This is a non-uniform sampling with a timing error of up to $\Delta t = 20\text{ }\mu\text{s}$, i.e. a phase error of up to $2\pi \cdot 6.5\text{ kHz} \cdot 20\text{ }\mu\text{s} = 0.82\text{ rad} = 47^\circ$ – a strong, deterministic distortion (spurious spectral lines). The Nyquist frequency at 25 kHz is 12.5 kHz, above both knock modes (6.5, 10.5 kHz) and the valve frequency (8 kHz), so no aliasing of signal components occurs. The chip noise is white with variance σ^2 at 50 kHz; decimation by 2 folds the band 12.5–25 kHz onto 0–12.5 kHz. White noise stays white with the same variance σ^2 , only its PSD per Hz doubles – nothing is lost that a band-pass could have rejected anyway. (A real ECU would use an analog anti-aliasing filter, Sheet 5.)

b) Transition width $\Delta f = 1\text{ kHz}$, $\Delta F = 0.04$, $\Delta\omega = 0.2513$. Hamming: $N \approx 3.3/0.04 = 82.5 \Rightarrow N = 83$ (odd, so that the delay is an integer and the filter is of type I). Kaiser: $\beta = 0.5842 \cdot 19^{0.4} + 0.07886 \cdot 19 = 3.395$, $N - 1 \approx 32/(2.285 \cdot 0.2513) = 55.7$, $N = 57$. The ideal cut-offs lie in the middle of the transition bands: $f_{c1} = 5.5\text{ kHz}$, $f_{c2} = 7.5\text{ kHz}$, because the window smears the ideal edge symmetrically over the transition band.

c) The ideal band-pass is the difference of two ideal low-passes:

$$h_d[m] = \frac{1}{2\pi} \int_{\omega_{c1} \leq |\omega| \leq \omega_{c2}} e^{j\omega m} d\omega = \frac{\sin(\omega_{c2}m) - \sin(\omega_{c1}m)}{\pi m}, \quad h_d[0] = \frac{\omega_{c2} - \omega_{c1}}{\pi}.$$

It is infinitely long and non-causal; we truncate it to $|m| \leq (N - 1)/2$, multiply with the window and shift by $(N - 1)/2$: $h[n] = h_d[n - \frac{N-1}{2}] w[n]$. $h[n]$ is symmetric, hence linear phase with

a constant group delay of $(N - 1)/2 = 41$ samples = 1.64 ms. At 3000 rpm (18000°/s) this is 29.5°, at 6000 rpm 59° crank angle! The filter output belonging to a 10°–70° ATDC window appears up to 59° later; the evaluation window must be shifted by the (rpm dependent) filter delay, otherwise most of the knock energy is missed (see 10.3).

d) Pre-warped edges (rad/s): $\Omega(5 \text{ kHz}) = 36327$, $\Omega(6 \text{ kHz}) = 46953$, $\Omega(7 \text{ kHz}) = 60440$, $\Omega(8 \text{ kHz}) = 78787$; $\Omega_0 = \sqrt{46953 \cdot 60440} = 53271$ (maps back to 6.502 kHz), $B = 13486$. $\lambda(5 \text{ kHz}) = 3.099$, $\lambda(8 \text{ kHz}) = 3.171$, so $\lambda_s = 3.099$. Butterworth:

$$n \geq \frac{\log_{10}[(10^{A_s/10} - 1)/(10^{A_p/10} - 1)]}{2 \log_{10} \lambda_s} = \frac{\log_{10}(9999/0.2589)}{2 \log_{10} 3.099} = \frac{4.587}{0.983} = 4.67 \Rightarrow n = 5.$$

The band-pass has order $2n = 10$, i.e. 5 biquads. Cost: 5 biquads with $b_1 = 0$ need $5 \cdot 4 = 20$ (general: 25) multiplications per sample versus 83 for the FIR filter (or 41 if the symmetry $h[n] = h[N - 1 - n]$ is exploited).

e) The burst $Ae^{-t/\tau} \sin(2\pi f_1 t)$, $t \geq 0$, has the spectrum magnitude $|X(f)| \approx \frac{A}{2} / \sqrt{(1/\tau)^2 + (2\pi(f - f_1))^2}$ near f_1 ; $|X|^2$ drops by 3 dB where $2\pi|f - f_1| = 1/\tau$, so the full -3 dB width is $B_k = 1/(\pi\tau) = 398 \text{ Hz}$. A filter matched to the burst has the same shape: a narrower band-pass rejects more noise but rings longer than the burst and starts late, a wider one passes more noise. A second-order band-pass with $B = 400 \text{ Hz}$ has exactly the decay time of the knock burst.

Pre-warping: $\Omega_0 = 2f_s \tan(\pi \cdot 0.26) = 53245 \text{ rad/s}$, $c = 2f_s = 50000$, $B \approx 2\pi \cdot 400 / \cos^2(0.8168) = 5363 \text{ rad/s}$. Inserting $s = c(1 - z^{-1})/(1 + z^{-1})$ and multiplying by $(1 + z^{-1})^2$:

$$H(z) = \frac{Bc(1 - z^{-2})}{(c^2 + Bc + \Omega_0^2) + 2(\Omega_0^2 - c^2)z^{-1} + (c^2 - Bc + \Omega_0^2)z^{-2}}.$$

With $d = c^2 + Bc + \Omega_0^2$: $b_0 = Bc/d = 0.04786$, $b_1 = 0$, $b_2 = -b_0$, $a_1 = 2(\Omega_0^2 - c^2)/d = 0.11957$, $a_2 = (c^2 - Bc + \Omega_0^2)/d = 0.90428$. (The tool of 10.2, which places the edges exactly, gives $b_0 = 0.047898$, $a_1 = 0.119566$, $a_2 = 0.904204$.) Poles: $r = \sqrt{a_2} = 0.9509$, angle $\arccos(-a_1/2r) = 93.6^\circ \hat{=} 6.5 \text{ kHz}$. Decay time of $h[n] \propto r^n$: $\tau_f = -T_s / \ln r = 0.80 \text{ ms}$ – the filter “rings” exactly like the knock burst. Note that at $f_s = 25 \text{ kHz}$ the knock poles lie close to $\pm j$, so $|a_1|$ is small.

Exercise 10.2 Filter design tool

JSLINUX

Files in `code/jslinux/sheet10/`: `filtdesign.h` (design library), `filterdesign.c` (design, check, export), `coefquant.c` (word-length study), `filtertest.c` (test on the virtual engine).

```
1 gcc -O2 -o filterdesign filterdesign.c -lm && ./filterdesign # writes knock_filter.h
2 gcc -O2 -o coefquant coefquant.c -lm && ./coefquant && ./coefquant 200000
3 gcc -O2 -o filtertest filtertest.c -lm && ./filtertest
```

- FIR window method.* Complete `fd_window()` (Hann, Hamming, Blackman, Kaiser) and `fd_fir_bandpass()`. `filterdesign` prints, for $N = 83$ and every window, the worst pass-band loss and stop-band gain, and the smallest odd N that meets the specification. Explain the ranking of the windows. Is the Kaiser estimate of 10.1 b) met?
- IIR Butterworth band-pass.* Complete `fd_prewarp()` and the pole transformation in `fd_butter_bandpass()`: every prototype pole p gives the band-pass poles $s = \frac{1}{2}(pB \pm \sqrt{p^2 B^2 - 4\Omega_0^2})$, which are mapped with $z = (2f_s + s)/(2f_s - s)$. Check the specification, list the pole radii and frequencies of the sections, and compare the group delay at 6.5 kHz with the FIR filter. What goes wrong without pre-warping?
- Complete `fd_bp2()` and compare with your hand calculation of 10.1 e). Report the attenuation of the 400-Hz and a 1000-Hz version at 8 kHz and 10.5 kHz.

- d) *Export.* Complete the Q15/Q2.14 conversion (rounding, saturation) and generate `knock_filter.h`. Why are the biquad coefficients exported in Q2.14 rather than in Q15? Why are the sections scaled to a peak gain of 1 (L_∞ scaling) and ordered by pole radius?
- e) *Coefficient quantisation.* Complete `quant_vec()` and the quantisation of a_1, a_2 in `coefquant.c`. The program realises the filter (1) as a biquad cascade and (2) as one 10th-order direct-form polynomial, quantises the coefficients to $B = 8 \dots 32$ bits and reports the pole displacement, the largest pole radius, the pass-band deviation and the stop-band gain. Interpret the results for $f_s = 25$ kHz and for the same analog specification at $f_s = 200$ kHz. Which word length do you need for the cascade, which for the direct form?
- f) *Test on the engine signal.* Complete `sos_step()` (TDF II, float) and `sosq_step()` (DF I, Q15 data, Q2.14 coefficients, 32-bit accumulator, rounding, saturation) in `filtertest.c`. Compare the window energies with and without knock for FIR and IIR, the delay of the filtered burst, and the error of the Q15 implementation.

Solution

a) Core of the FIR design (code/solutions/jslinux/sheet10/filttdesign.h):

```
1 FD_STATIC void fd_fir_bandpass(double *h, int N, double f1, double f2, double fs,
2                               int win, double beta)
3 {
4     double w1 = 2.0 * M_PI * f1 / fs, w2 = 2.0 * M_PI * f2 / fs, g;
5     int n;
6     for (n = 0; n < N; n++) {
7         double m = n - 0.5 * (N - 1);
8         double hd = (fabs(m) < 1e-9) ? (w2 - w1) / M_PI
9                                   : (sin(w2 * m) - sin(w1 * m)) / (M_PI * m);
10        h[n] = hd * fd_window(win, n, N, beta);
11    }
12    g = c_abs(fd_fir_resp(h, N, 0.5 * (f1 + f2), fs));
13    for (n = 0; n < N; n++) h[n] /= g;
14 }
```

Output of `filterdesign` (cut-offs 5.5/7.5 kHz):

window ($N = 83$)	pass min [dB]	pass max [dB]	stop max [dB]	min. N meeting spec
rectangular	-0.798	0.004	-25.0	> 201
Hann	0.000	0.076	-44.0	77
Hamming	-0.026	0.016	-52.9	77
Blackman	-0.301	0.001	-29.3	103
Kaiser ($\beta = 3.395$)	-0.154	0.016	-41.2	63

The rectangular window has the narrowest main lobe but side lobes of only ≈ -21 dB (Gibbs), it never reaches 40 dB. Hann and Hamming have a main lobe twice as wide and side lobes of $-31/-41$ dB; their stop-band attenuation ($\approx 44/53$ dB) is sufficient. Blackman has -58 dB side lobes but the widest main lobe ($\approx 5.5/N$): at $N = 83$ the transition band is too wide and the stop-band edge is violated (-29 dB at 5 kHz); it needs $N = 103$. The Kaiser window trades main-lobe width against side-lobe level by β and reaches the specification with the shortest filter, $N = 63$; the empirical formula ($N = 57$) is slightly optimistic because it ignores the pass-band ripple and the grid evaluation. We keep the Hamming design with $N = 83$ (comfortable margin).

b) Pole transformation (code/solutions/jslinux/sheet10/filttdesign.h):

```
1     for (k = 0; k < n; k++) {
2         cplx p = c_scale(c_expj(M_PI * (2.0 * k + n + 1) / (2.0 * n)), rp);
3         cplx pB, disc, sq, s1, s2;
4         if (p.im < -1e-12) continue; /* use upper half + real */
5         pB = c_scale(p, B);
6         disc = c_sub(c_mul(pB, pB), c_make(4.0 * W0 * W0, 0.0));
```

```

7      sq = c_sqrt(disc);
8      s1 = c_scale(c_add(pB, sq), 0.5);
9      s2 = c_scale(c_sub(pB, sq), 0.5);
10     if (fabs(p.im) < 1e-12) {                /* real prototype pole: */
11         zp[ns++] = fd_bilinear(s1.im >= 0 ? s1 : s2, fs); /* one conj pair */
12     } else {                                  /* complex: two pairs */
13         zp[ns++] = fd_bilinear(s1, fs);
14         zp[ns++] = fd_bilinear(s2, fs);
15     }
16 }

```

Result ($\lambda_s = 3.099$, $n = 5$, 5 biquads, each $b = [b_0, 0, -b_0]$):

section	b_0	a_1	a_2	$ p $	f_p [Hz]
0	0.12634	0.11059	0.74733	0.8645	6505
1	0.10456	-0.03989	0.79089	0.8893	6161
2	0.10355	0.26523	0.79290	0.8905	6845
3	0.04236	-0.14173	0.91528	0.9567	5955
4	0.68429	0.37993	0.91667	0.9574	7045

Check: pass band $-1.000 \dots 0$ dB, stop band ≤ -43.3 dB – specification met. Attenuation at 8 kHz: FIR 52.9 dB, IIR 44.3 dB; at 10.5 kHz: FIR 69.5 dB, IIR 106.7 dB (the IIR filter has 5-fold zeros at $z = -1$). Group delay at 6.5 kHz: IIR 22.4 samples (0.90 ms), FIR 41 samples (1.64 ms). The IIR filter is faster and needs 20 instead of 83 multiplications, but its phase is non-linear (group delay largest near the band edges) – irrelevant for an energy detector. Without pre-warping the analog edges $2\pi f$ are mapped by the bilinear transform to $f' = \frac{f_s}{\pi} \arctan(\pi f / f_s)$: 6 kHz $\rightarrow$ 5.09 kHz, 7 kHz $\rightarrow$ 5.69 kHz. The band would be shifted and compressed by about 1 kHz, i.e. the knock frequency would lie outside the pass band.

c) **fd_bp2**: $B = 400$ Hz: $b_0 = 0.047898$, $a_1 = 0.119566$, $a_2 = 0.904204$, $|p| = 0.95090$, peak at 6500 Hz, attenuation 18.1 dB at 8 kHz, 30.5 dB at 10.5 kHz, 38.3 dB at 1 kHz, group delay 19.9 samples. $B = 1000$ Hz: $b_0 = 0.112160$, $a_1 = 0.111496$, $a_2 = 0.775680$, $|p| = 0.8807$; only 10.4 dB at 8 kHz and 22.5 dB at 10.5 kHz, group delay 7.9 samples. A single biquad cannot meet the 40 dB specification; it is the cheap option for the Uno.

d) Q15 covers $[-1, 1)$ only, but biquad coefficients satisfy $|a_1| < 2$ ($a_1 = -2r \cos \theta$) and, after gain scaling, b_0 may exceed 1 as well. Q2.14 (one more integer bit) covers $[-2, 2)$ for all stable biquads. With Q15 data the product is Q29; shifting the 32-bit accumulator by 14 gives Q15. (For our filter at $f_s = 25$ kHz all coefficients happen to be < 1 because the poles lie near $\pm j$, but the header format must work for any design.) L_∞ scaling: every section has peak gain 1, so for sinusoidal inputs no intermediate signal exceeds the input amplitude – the condition for overflow-free fixed-point processing. Ordering by pole radius puts the most resonant sections (largest gain peaks, longest ringing, most sensitive to rounding noise) at the end, where the signal is already band-limited.

e) Output of **coefquant** for $f_s = 25$ kHz (max $|a_k|$ of the direct form: 7.03):

B	biquad cascade, Q2.($B - 2$)				direct form, order 10			
	max $ \Delta p $	max $ p $	pass dev	stop	max $ \Delta p $	max $ p $	pass dev	stop
8	4.8e-3	0.9601	0.68	-44.3	0.29	1.172	unstable	
10	1.1e-3	0.9581	0.38	-43.0	0.17	1.080	unstable	
12	2.0e-4	0.9576	0.06	-43.2	0.16	1.058	unstable	
14	7.8e-5	0.9575	0.01	-43.2	0.091	1.000	unstable	
16	1.6e-5	0.9574	0.00	-43.3	0.050	0.9623	1.99	-43.4
20	1.0e-6	0.9574	0.00	-43.3	3.5e-3	0.9579	0.10	-43.2
24	6.6e-8	0.9574	0.00	-43.3	7.1e-5	0.9574	0.00	-43.3

(pass dev = worst deviation in dB from the exact response in the pass band; stop = worst stop-band gain in dB.) The pole displacement of the cascade scales like the quantisation step

$2^{-(B-2)}$; already 10–12 bits are enough, 16 bits (Q2.14) are exact for all practical purposes. The direct form is unstable up to 14 bits and still shows 2 dB error at 16 bits: the roots of a high-order polynomial with clustered roots are extremely sensitive to its coefficients ($\partial p_i / \partial a_k = p_i^{N-k} / \prod_{j \neq i} (p_i - p_j)$, and the poles of a narrow band-pass are close to each other). At $f_s = 200$ kHz (the same analog filter, now $n = 6$, 12th order) the poles crowd near $z = 1$ (angle 10.7° , radius 0.9958): the cascade needs about 14 bits for 0.3 dB accuracy (12 bits: 1.8 dB, 8 bits: 25 dB error), and the direct form ($\max |a_k| = 806$) is unstable at *every* tested word length, even with 32 bits (and even in double precision its frequency response cannot be evaluated accurately). Lessons: always use second-order sections, and do not oversample a narrow-band IIR filter unnecessarily. For the single biquad ($B = 400$ Hz, $f_s = 25$ kHz) 8 bits already give $r = 0.95197$ instead of 0.95090 and a shift of 11 Hz; at $f_s = 200$ kHz 8 bits move the peak to 5651 Hz, 6 bits put the pole on the unit circle.

f) TDF II (float) and DFI (Q15) in code/solutions/jslinux/sheet10/filtertest.c:

```

1 static float sos_step(sos_t *f, float x)
2 {
3     int k;
4     for (k = 0; k < KF_NSEC; k++) {
5         const float *c = kf_sos[k];
6         float y = c[0] * x + f->s1[k];
7         f->s1[k] = c[1] * x - c[3] * y + f->s2[k];
8         f->s2[k] = c[2] * x - c[4] * y;
9         x = y;
10    }
11    return x;
12 }
13
14 /* ---- Q15 biquad cascade, direct form I, saturating ----- */
15 typedef struct { int16_t x1[KF_NSEC], x2[KF_NSEC], y1[KF_NSEC], y2[KF_NSEC]; long nsat; }
16   sosq_t;
17 static int16_t sat16(int32_t v, long *nsat)
18 {
19     if (v > 32767) { (*nsat)++; return 32767; }
20     if (v < -32768) { (*nsat)++; return -32768; }
21     return (int16_t)v;
22 }
23 static int16_t sosq_step(sosq_t *f, int16_t x)
24 {
25     int k;
26     for (k = 0; k < KF_NSEC; k++) {
27         const int16_t *c = kf_sos_q14[k];
28         int32_t acc = (int32_t)c[0] * x + (int32_t)c[1] * f->x1[k] + (int32_t)c[2] * f->x2[k]
29                     - (int32_t)c[3] * f->y1[k] - (int32_t)c[4] * f->y2[k];
30         int16_t y = sat16((acc + (1 << 13)) >> 14, &f->nsat); /* Q29 -> Q15, rounded */
31         f->x2[k] = f->x1[k]; f->x1[k] = x;
32         f->y2[k] = f->y1[k]; f->y1[k] = y;
33         x = y;
34     }
35     return x;
36 }

```

Output (3000 rpm, knock amplitude 0.3 V, onset 15° ATDC, noise 0.05 V):

window energy 10° – 70° ATDC [V^2]	knock	no knock	ratio
FIR (float, $N = 83$)	0.379	0.0199	19.0
IIR biquads (float, 5 sections)	0.327	0.0116	28.1
IIR biquads (Q15/Q2.14)	0.327	0.0116	28.2

The IIR filter separates knock and noise better: its pass band is narrower (noise bandwidth 1.16 kHz versus ≈ 2 kHz for the FIR filter whose -6 dB points are 5.5/7.5 kHz) and its delay is

smaller, so more of the burst falls into the window. Peak of the filtered burst after the knock onset: FIR 1.92 ms, IIR 1.44 ms (group delay plus build-up). The Q15 implementation deviates from the float filter by $5.6 \cdot 10^{-3}$ (−45 dB) relative rms error; no saturation occurred. The error is dominated by the rounding noise of the last, most resonant sections.

Exercise 10.3 Real-time filtering on the microcontroller

WOKWI

- a) *ESP32, float*. Open `code/wokwi/sheet10_esp32_filter/` and replace the placeholder `knock_filter.h` by your generated header. The sketch samples the KNOCK pin at 25 kHz (deadline loop) and records 300 samples after each TDC of cylinder 1 (TDC pulse while CAM is HIGH). Complete `fir_step()` (circular buffer stored twice, so the newest N samples are always contiguous) and `iir_step()` (TDF II).
- i. Run at 3000 rpm with knock probability 100 % and 0 %. Copy one record into JSLinux and plot x , y_{FIR} , y_{IIR} and the KNK flag. Compare the window energies printed in the `# rec` lines.
 - ii. Repeat at 6000 rpm. Explain the drastic change of the FIR window energy and propose a fix.
 - iii. Why does the sketch use `float` and not `double` on the ESP32? Compare the indicative benchmark of FIR and IIR with the operation counts.
- b) *Uno, Q15*. Open `code/wokwi/sheet10_uno_q15/`. Timer1 interrupts at 25 kHz; the ISR reads the ADC, starts the next conversion and filters with the single biquad `kf_bp2_q14`.
- i. Complete `biquad_q15()`: DFI, 32-bit accumulator (Q29), rounding, conversion to Q15, overflow detection, saturation or wrap-around (`SATURATE`). Verify it in `MODE 0`, where a recorded block is filtered with float and with Q15 arithmetic.
 - ii. Cycle budget: how many CPU cycles are available per sample? Measure the ISR duration (column `isr_cycles`, Timer1 count at the end of the ISR, or the pulse width on D8 with the logic analyzer). How many biquads, how many FIR taps would fit? Could the 10th-order cascade of 10.2 run on the Uno at 25 kHz?
 - iii. Scaling: the input is mapped by $x = (\text{adc} - 512) \cdot 2^{\text{SHIFT}}$. Which `SHIFT` uses the full Q15 range? Show that the band-pass output can still overflow when the *input* clips (hint: a clipped sine approaches a square wave). Run `MODE 1` with knock amplitude 3 V and `SHIFT = 6, 8, 10` and compare saturation with wrap-around.
 - iv. Complete the window energy `energy += y*y >> 8`. Show that the 32-bit accumulator cannot overflow down to 600 rpm. Why is $y^2 \gg 8$ much cheaper on the AVR than $y^2 \gg 12$?

Solution

a) FIR with doubled circular buffer and cascade in TDF II (code/solutions/wokwi/sheet10_esp32_filter/sketch.ino):

```

1 static float fir_step(float x)
2 {
3     fir_buf[fir_pos] = x;
4     fir_buf[fir_pos + KF_FIR_N] = x;
5     if (++fir_pos == KF_FIR_N) fir_pos = 0;
6     const float *p = &fir_buf[fir_pos + KF_FIR_N - 1];    /* newest */
7     float acc = 0.0f;
8     for (int k = 0; k < KF_FIR_N; k++) acc += kf_fir[k] * p[-k];
9     return acc;
10 }
11
12 /* ----- IIR: biquad cascade, transposed direct form II ---- */
13 static float s1[KF_NSEC], s2[KF_NSEC];

```

```

14
15 static float iir_step(float x)
16 {
17     for (int k = 0; k < KF_NSEC; k++) {
18         const float *c = kf_sos[k];          /* b0 b1 b2 a1 a2 */
19         float y = c[0] * x + s1[k];
20         s1[k] = c[1] * x - c[3] * y + s2[k];
21         s2[k] = c[2] * x - c[4] * y;
22         x = y;
23     }
24     return x;
25 }

```

i. The following values were obtained by running the sketch against the chip model in a PC test harness; Wokwi gives different random realisations but the same picture. 3000 rpm, knock probability 100 %, knock amplitude 0.5 V: window 10° – 70° = samples 13–97, $E_{\text{FIR}} = 0.45 \dots 1.86$, $E_{\text{IIR}} = 0.43 \dots 1.79 \text{ V}^2$. Without knock: $E_{\text{FIR}} = 0.024 \dots 0.040$, $E_{\text{IIR}} = 0.015 \dots 0.027 \text{ V}^2$. In the plot the IIR output rises about 0.5 ms earlier than the FIR output; the KNK flag is HIGH for $3\tau = 2.4 \text{ ms}$ from the knock onset.

ii. 6000 rpm, knock amplitude 1 V: window = samples 6–48 (1.67 ms); $E_{\text{FIR}} = 0.02 \dots 0.51$, $E_{\text{IIR}} = 0.81 \dots 5.2 \text{ V}^2$. The FIR delay of 41 samples equals 59° at 6000 rpm: the filtered burst leaves the window before it has fully arrived. Fix: shift the window by the group delay, i.e. evaluate samples $[n_{10} + 41, n_{70} + 41]$ of the FIR output (equivalently $[10^\circ + \Delta\theta, 70^\circ + \Delta\theta]$ with $\Delta\theta = \tau_g\omega$), or use the IIR filter with half the delay – and shift its window by ≈ 22 samples as well.

iii. The ESP32 FPU supports single precision only; `double` is emulated in software and is roughly an order of magnitude slower. For a 16-bit-class signal (12-bit ADC) float (24-bit mantissa) is more than sufficient. Per sample the FIR needs 83 multiply-adds, the cascade 25 multiplications (20 non-zero); the benchmark should show roughly this ratio (Wokwi timing is only indicative). At 240 MHz even the FIR uses only a few percent of the $40 \mu\text{s}$ sampling period.

b) i. Q15 biquad (code/solutions/wokwi/sheet10_uno_q15/sketch.ino):

```

1 static inline int16_t biquad_q15(int16_t x)
2 {
3     /* kf_bp2_q14 = {b0, b1, b2, a1, a2}, b1 = 0 for the band-pass */
4     int32_t acc = (int32_t)kf_bp2_q14[0] * x
5                 + (int32_t)kf_bp2_q14[2] * bx2
6                 - (int32_t)kf_bp2_q14[3] * by1
7                 - (int32_t)kf_bp2_q14[4] * by2;    /* Q29 */
8     acc += 1L << 13;                               /* rounding */
9     /* Q29 -> Q15: bits 14..29 of acc. (acc << 2) >> 16 needs only byte
10      moves on the AVR, a loop ">> 14" would cost ~100 cycles. */
11     int16_t y = (int16_t)(((uint32_t)acc << 2) >> 16); /* wraps around */
12     if (acc >= (1L << 29) || acc < -(1L << 29)) {    /* |y| > 1: overflow */
13         n_ovf++;
14     #if SATURATE
15         y = acc > 0 ? 32767 : -32768;
16     #endif
17     }
18     bx2 = bx1; bx1 = x;
19     by2 = by1; by1 = y;
20     return y;
21 }

```

Since $b_1 = 0$ only 4 multiplications are needed. Block test (MODE 0, 400 samples with knock): maximum deviation from the float filter 1.8 LSB (Q15), rms 0.58 LSB – the rounding noise of the output, slightly amplified by the recursive part.

ii. $16 \text{ MHz}/25 \text{ kHz} = 640$ cycles per sample, shared by the ISR, the serial interrupt and `loop()`. From the disassembly the ISR has about 180 instructions (of which ≈ 100 cycles are register

save/restore) plus 5 calls of the $16 \times 16 \rightarrow 32$ multiplication helper of ≈ 40 cycles each – an estimate of 450–500 cycles, i.e. about 75% CPU load; measure it with `isr_cycles` or the D8 pulse. Each additional biquad costs $\approx 4 \cdot 40 + 50 \approx 200$ cycles, each FIR tap ≈ 45 cycles: at most one biquad or ≈ 3 –4 FIR taps fit. The 5-section cascade (≈ 1000 cycles) or the 83-tap FIR (≈ 3700 cycles) are impossible at 25 kHz – one reason why the Uno is limited to the single matched biquad (and why production ECUs use DSP cores with single-cycle MAC).

iii. $|\text{adc} - 512| \leq 512 = 2^9$; `SHIFT` = 6 maps the ADC range onto the full Q15 range. The band-pass has peak gain 1, so for any *sinusoidal* input with $|x| < 1$ its output stays below 1. With a larger shift the input saturates; a strongly clipped sine approaches a square wave whose fundamental has the amplitude $\frac{4}{\pi} \approx 1.27$ – the narrow band-pass extracts this fundamental and overflows. In the test (knock slider 3 V, 12 combustions, knock in 8 of them): `SHIFT` = 6 and 8: no overflow (the decaying burst is too short for the filter to build up to the square-wave fundamental); `SHIFT` = 10: 2 overflow events. With saturation the affected combustion has the energy $9.13 \cdot 10^7$, with wrap-around $6.28 \cdot 10^7$: a wrapped sample jumps from +1 to –1, which adds a large broadband error and falsifies the result, whereas saturation only clips. Saturation is therefore mandatory; the shift should be chosen such that the largest expected signal does not clip (`SHIFT` = 6 here, the small signals are still resolved with 2^6 Q15 units per ADC LSB).
 iv. $|y| \leq 2^{15}$, so $y^2 \gg 8 \leq 2^{22}$. At 600 rpm 180° take 50 ms = 1250 samples, the window (60°) 417 samples: $417 \cdot 2^{22} = 1.75 \cdot 10^9 < 2^{32}$. A shift by 8 (or 16) bits is a byte move on the 8-bit AVR; a shift by 12 is compiled as a loop of 12 four-byte shifts (≈ 80 cycles).

Deliverables

10.1 on paper. `knock_filter.h` and the output of `filterdesign` and `coefquant` (both sampling rates); plot of one ESP32 record (x , y_{FIR} , y_{IIR} , KNK) at 3000 and 6000 rpm; Uno: block-test error, measured ISR cycles, table of overflow events for the three shifts with a short discussion.